Solution: Triplet Sum Close to Target

Problem Statement

Given an array of unsorted numbers and a target number, find a **triplet in the array whose sum is as close to the target number as possible**, return the sum of the triplet. If there are more than one such triplet, return the sum of the triplet with the smallest sum.

Example 1:

```
Input: [-1, 0, 2, 3], target=3
Output: 2
Explanation: There are two triplets with distance '1' from the target:
[-1, 0, 3] & [-1, 2, 3]. Between these two triplets, the correct answer
will be [-1, 0, 3] as it has a sum '2' which is less than the sum of the
other triplet which is '4'. This is because of the following
requirement: 'If there are more than one such triplet, return the sum of
the triplet with the smallest sum.'
```

Example 2:

```
Input: [-3, -1, 1, 2], target=1
Output: 0
Explanation: The triplet [-3, 1, 2] has the closest sum to the target.
```

Example 3:

```
Input: [1, 0, 1, 1], target=100
Output: 3
Explanation: The triplet [1, 1, 1] has the closest sum to the target.
```

Example 4:

```
Input: [0, 0, 1, 1, 2, 6], target=5
Output: 4
Explanation: There are two triplets with distance '1' from target: [1,
1, 2] & [0, 0, 6]. Between these two triplets, the correct answer will
be [1, 1, 2] as it has a sum '4' which is less than the sum of the other
triplet which is '6'. This is because of the following requirement: 'If
there are more than one such triplet, return the sum of the triplet with
the smallest sum.'
```

Solution

This problem follows the **Two Pointers** pattern and is quite similar to "Triplet Sum to Zero".

We can follow a similar approach to iterate through the array, taking one number at a time. At every step, we will save the difference between the triplet and the target number, so that in the end, we can return the triplet with the closest sum.

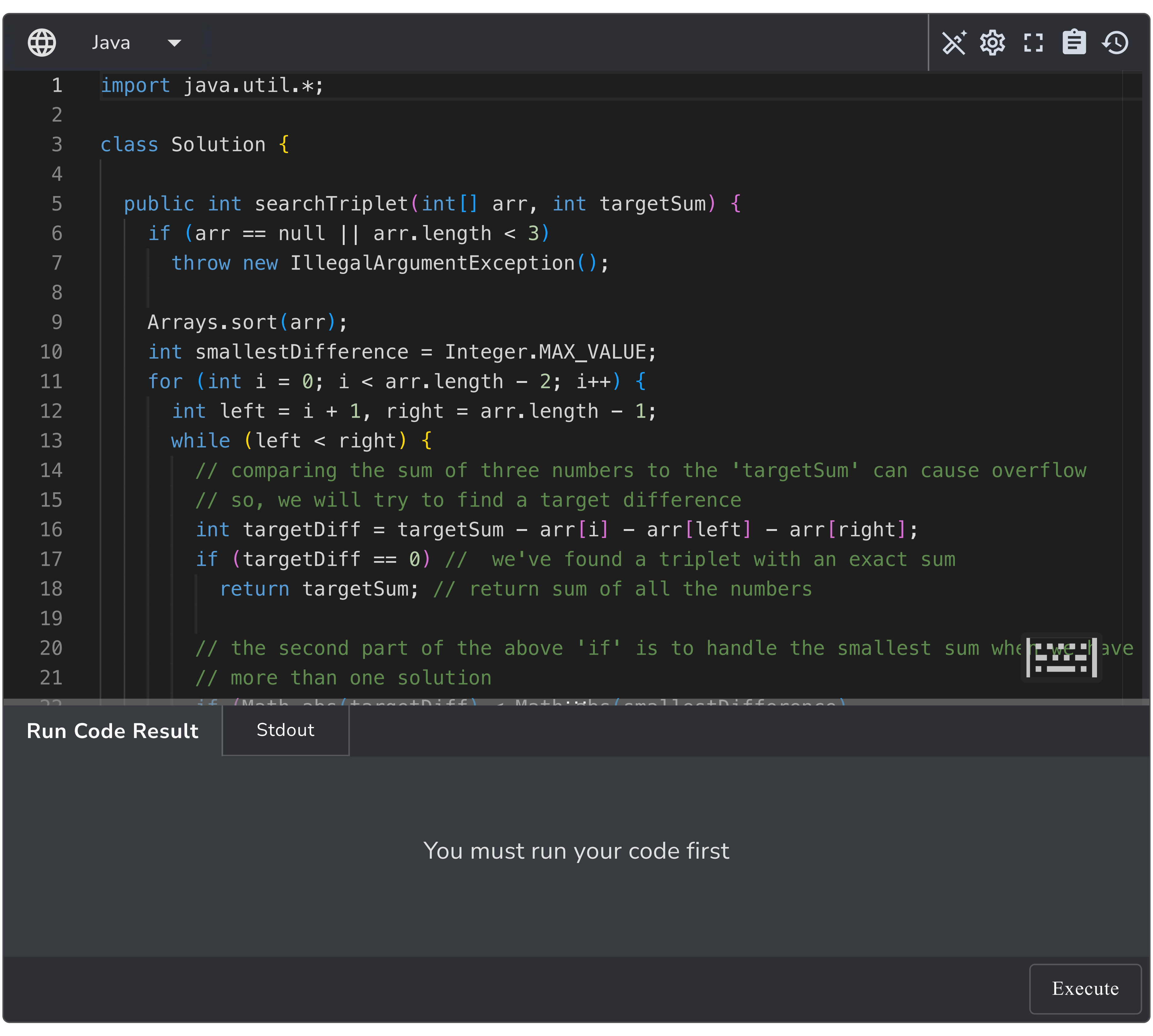
Here's a detailed walkthrough of the algorithm:

1. Initially, the method checks whether the input array `arr` is `null` or its length is less than 3. If either condition is true, the method throws an `IllegalArgumentException`, as it is impossible to find a triplet in these cases.
2. The input array `arr` is then sorted in ascending order. Sorting is important as it allows us to move our pointers based on the sum we are getting and how close we are to the target sum.
3. The `smallestDifference` variable is initialized to `Integer.MAX_VALUE`, which will keep track of the smallest difference we have found so far between our target sum and the sum of our current triplet.
4. The function then iterates through `arr` using a `for` loop, stopping when it is two positions from the end of `arr` (`arr.length - 2`). This is because we are always looking for triplets and thus don't need to consider the last two positions in this loop.
5. Inside the `for` loop, two pointers, `left` and `right`, are initialized. `left` is set to `i + 1` (one position to the right of our current position) and `right` is set to the last index of the array (`arr.length - 1`).
6. We start a `while` that continues as long as `left` is less than `right`. Inside this loop, we calculate the difference between the target sum and the sum of the numbers at our current positions in the array (`targetDiff`). This allows us to see how close the current triplet sum is to our target sum.
7. If `targetDiff` equals 0, that means the sum of our current triplet exactly matches the target sum, and we return the `targetSum` immediately as our result.
8. Otherwise, we check if the absolute value of `targetDiff` is less than the absolute value of `smallestDifference` (meaning we've found a closer sum), or if it's equal but `targetDiff` is greater (meaning it's a larger sum that is equally close). If either condition is true, we update `smallestDifference` with `targetDiff`.
9. Next, we check if `targetDiff` is greater than 0. If it is, we increment `left` to try and increase our current triplet sum (since the array is sorted, moving `left` to the right will increase the sum). If `targetDiff` is not greater than 0, we decrement `right` to decrease our triplet sum.
10. This `while` loop continues until `left` and `right` cross, at which point we have examined all possible triplets for our current value of `i`.
11. The `for` loop continues until we have tried every possible starting point for our triplet.
12. Once all possible triplets have been considered, the function returns `targetSum - smallestDifference`. This is the sum of the triplet that was closest to our target sum.

let's visualize the example 4 via below diagram.

Code

Here is what our algorithm will look like:



```java
import java.util.*;

class Solution {

  public int searchTriplet(int[] arr, int targetSum) {
    if (arr == null || arr.length < 3)
      throw new IllegalArgumentException();

    Arrays.sort(arr);
    int smallestDifference = Integer.MAX_VALUE;
    for (int i = 0; i < arr.length - 2; i++) {
      int left = i + 1, right = arr.length - 1;
      while (left < right) {
        // comparing the sum of three numbers to the 'targetSum' can cause overflow
        // so, we will try to find a target difference
        int targetDiff = targetSum - arr[i] - arr[left] - arr[right];
        if (targetDiff == 0) //  we've found a triplet with an exact sum
          return targetSum; // return sum of all the numbers

        // the second part of the above 'if' is to handle the smallest sum when we have
        // more than one solution
```

Time Complexity

Sorting the array will take $O(N * logN)$. Overall, the function will take $O(N * logN + N^2)$, which is asymptotically equivalent to $O(N^2)$.

Space Complexity

The above algorithm's space complexity will be $O(N)$, which is required for sorting.

Please wait! Loading...

← Previous
Triplet Sum Close To Target (Medium)

Next →
Triplets With Smaller Sum (Medium)

☐ Mark as Completed